## I. INTRODUCTION TO GRAPH DATA

Traditional Deep Learning models (like CNNs) excel at processing Euclidean data (grids or sequences). However, much of the world's data is **non-Euclidean** and is best represented as a **Graph**.

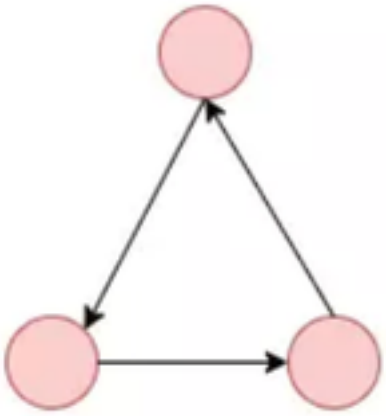
Graphs provide a powerful framework for modeling complex relationships in:

- **Social Networks:** Connections between users.
- **Transportation:** Nodes as cities and edges as roads.
- **Chemical Compounds:** Interactions between molecules.
- **Academic Libraries:** Research papers linked by citations (like your **50-file project**).

### 1.1. What is a Graph?

A Graph consists of two fundamental components:

- **Nodes (Vertices):** Represent entities (e.g., a person, a place, or an object).
- **Edges (Links):** Define the relationships or dependencies between nodes. Edges can be **Directed** (one-way) or **Undirected** (two-way).



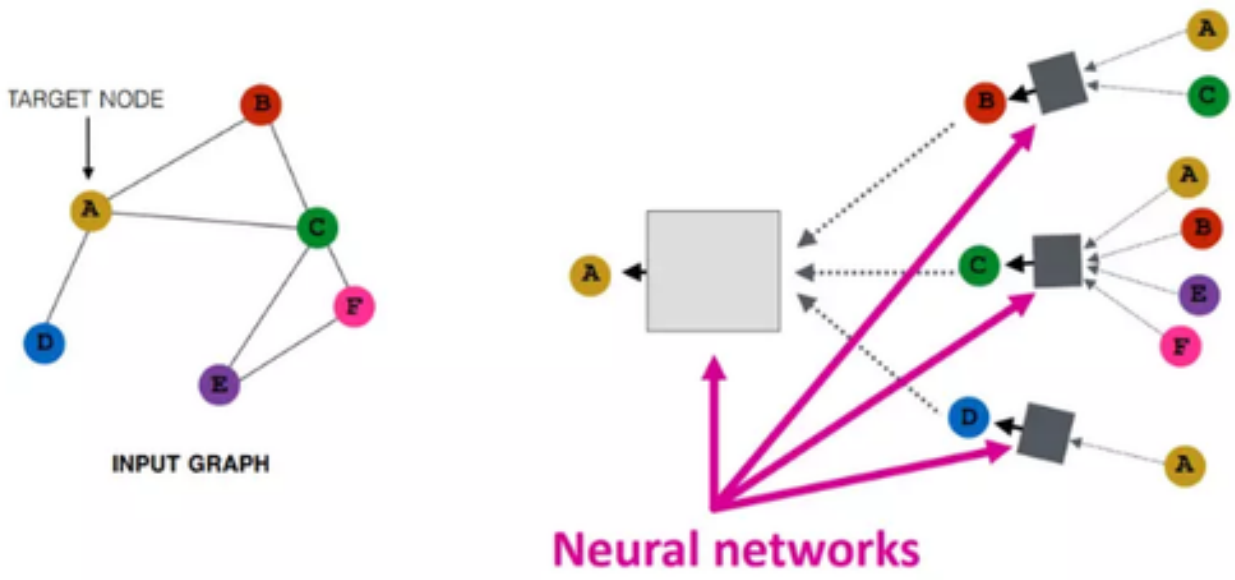
## II. CORE CONCEPTS OF GRAPH NEURAL NETWORKS (GNNs)

A GNN is a type of Neural Network specifically designed to operate on the graph structure. It captures information by passing messages through the vertices and edges.

### 2.1. How GNNs Work

GNNs maintain information about nodes, edges, and global context by converting the input graph into **Graph Embeddings**.

1. **Aggregation:** Each node collects feature vectors from its immediate neighbors.
2. **Combination:** The model updates the node's feature vector by combining its current data with the aggregated neighbor data.
3. **Propagation:** This process repeats across multiple layers, allowing information to flow throughout the graph.



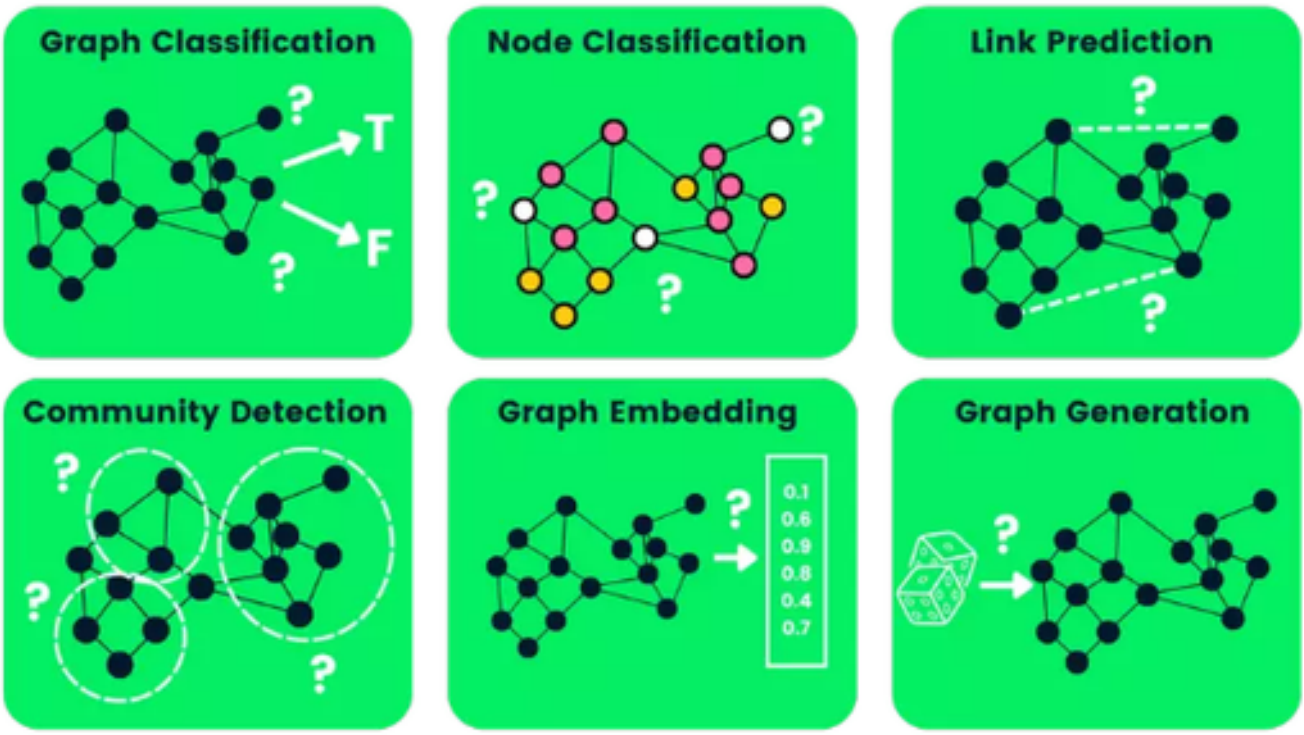

### 2.2. Common GNN Architectures

- **Graph Convolutional Networks (GCN):** The most popular type; it uses a convolution mechanism to aggregate neighbor information.
- **GraphSAGE:** Uses sampling and aggregation to handle extremely large graphs efficiently.
- **Graph Attention Networks (GAT):** Uses an **Attention Mechanism** to assign different weights (importance) to different neighbors.
- **Graph Autoencoders (GAE):** Used to learn unsupervised embeddings and discover hidden structures.

## III. KEY TASKS IN GNNs

GNNs are used to solve various predictive and structural problems:

1. **Node Classification:** Predicting the label of a missing node (e.g., categorizing a document).
2. **Link Prediction:** Predicting whether a relationship exists between two nodes (e.g., "People you may know").
3. **Graph Classification:** Categorizing an entire graph (e.g., identifying if a molecule is toxic).
4. **Community Detection:** Clustering nodes into groups based on edge density.
5. **Graph Embedding:** Mapping graphs into low-dimensional vectors for downstream tasks.




## IV. PRACTICAL IMPLEMENTATION (CORA DATASET)

In this example, we use the **Cora Planetoid** dataset—a citation network where nodes are research papers and edges are citations.

### 4.1. The GCN Architecture

A standard GCN for node classification typically consists of:

- **GCN Layers:** To propagate information.
- **ReLU Activation:** For non-linearity.
- **Dropout:** To prevent overfitting.

### 4.2. Mathematical Foundation

The update rule for a GCN layer is defined as:

$$x_v^{(\ell+1)} = W^{(\ell+1)} \sum_{w \in N(v) \cup \{v\}} \frac{1}{c_{w,v}} \cdot x_w^{(\ell)}$$

Where:

-

$x_v^{(\ell+1)}$

- : is the updated feature of node v.
- W : is the weight matrix.
- : is a normalization constant based on the graph structure.

## V. VISUALIZATION AND EVALUATION

Before training, nodes are often scattered randomly in the embedding space. After training for 100 epochs (using Adam optimizer and Cross-Entropy loss), the GNN learns to cluster similar nodes together.

- **Performance Metric:** Accuracy is calculated as the percentage of correct label predictions.
- **Visualization: t-SNE** is commonly used to project high-dimensional node embeddings into a 2D plot to observe clearly defined communities.